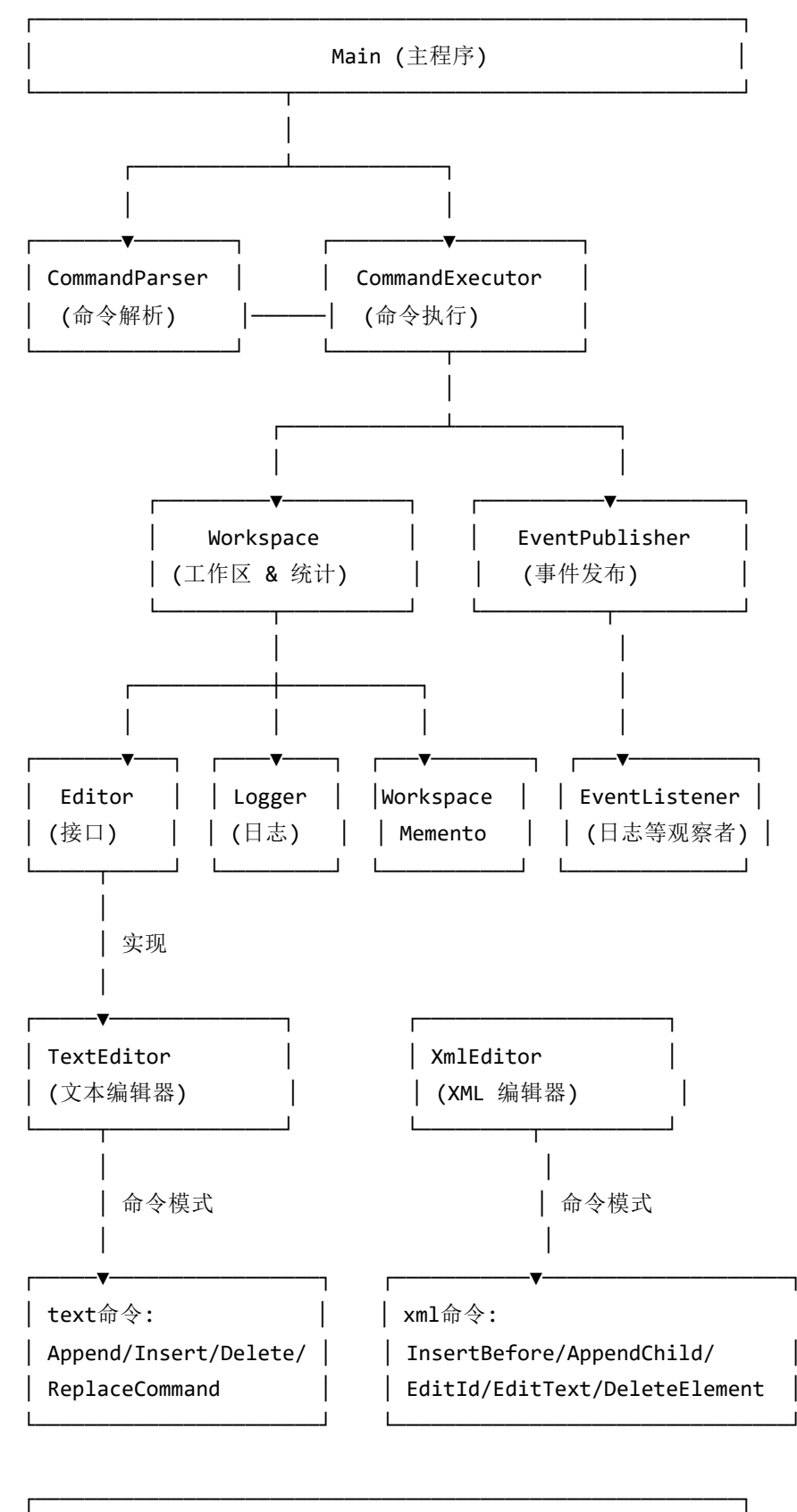


Lab2 架构文档

23307110043 孙天宇

2.1 系统架构

2.1.1 模块划分图



Statistics (统计模块)
(记录每个文件会话内编辑时长, 由 Workspace 驱动)

SpellCheck (拼写检查)
SpellChecker 接口 + LanguageToolSpellChecker 适配器
由 CommandExecutor 调用, 仅依赖接口

2.1.2 模块职责说明

- **Main (editor.Main)**
 - 程序入口, 循环读取用户命令。
 - 初始化 Workspace、CommandExecutor、Logger 监听等。
- **CommandParser (editor.command.CommandParser)**
 - 将用户输入字符串解析为结构化的 ParsedCommand。
 - 处理带引号文本、行:列格式、多种命令格式。
 - Lab2 新增解析:
 - init <text|xml> <file> [with-log]
 - insert-before / append-child / edit-id / edit-text / delete-element / xml-tree / spell-check
- **CommandExecutor (editor.command.CommandExecutor)**
 - 根据 ParsedCommand 分发并执行命令。
 - 调用 Workspace 进行文件管理与统计更新。
 - 针对不同类型 Editor 执行不同命令:
 - TextEditor : append/insert/delete/replace/show 等。
 - XmlEditor : insert-before/append-child/edit-id/edit-text/delete-element/xml-tree。
 - 集成拼写检查: 调用 SpellChecker 接口, 输出格式化错误报告。
- **Workspace (editor.workspace.Workspace)**
 - 管理多文件、多编辑器实例与全局状态。
 - 文件操作: loadFile/saveFile/saveAllFiles/forceCloseFile 等。
 - Lab2 扩展:
 - initFile(fileType, filePath, withLog) 支持 text/xml 初始化。
 - 加载文件时根据扩展名选择 TextEditor / XmlEditor。
 - 日志:
 - 文本: 检测 # log 自动启用日志。
 - XML: 检测根元素 log="true" 自动启用日志。
 - 状态持久化: 依赖 WorkspaceMemento。
 - 统计:
 - 持有 Statistics, 在 setActiveFile / forceCloseFile 中更新时长。
 - getFileList() 中为每个文件计算并返回可读时长字符串。

- **Editor 接口 (editor.editor.Editor)**

- 抽象所有编辑器的基本能力：
 - 文本操作： append/insert/delete/replace/show/getContent/load 。
 - 撤销/重做： executeCommand/undo/redo/canUndo/canRedo 。
 - 状态： isModified/markModified/clearModified 。
- Lab2 中由 TextEditor 与 XmlEditor 实现，支持多态扩展。

- **TextEditor (editor.editor.TextEditor)**

- 面向纯文本文件的编辑器。
- 使用行列表存储文本，支持行列级操作。
- 使用命令栈实现撤销/重做。
- 拼写检查时按行调用 SpellChecker 。

- **XmlEditor (editor.editor.XmlEditor)**

- 面向 XML 文件的编辑器，实现 Editor 接口。
- 维护：
 - XmlElement root：根节点。
 - Map<String, XmlElement> idMap：ID 映射。
- 提供 XML 操作：
 - insertBefore/appendChild/editId/editText/deleteElement/xmlTreeToString 。
- 负责 XML 解析/序列化，满足实验 XML 语法子集与约束（唯一 ID、不支持混合内容等）。

- **XmlElement (editor.editor.XmlElement)**

- 组合模式节点，表示 XML 树中的一个元素。
- 字段： tagName, id, text, attributes, children, parent 。
- 支持：
 - 添加/插入/删除子节点。
 - 属性增删改查。
 - 递归遍历（用于构建 ID 映射、树形打印、拼写检查等）。

- **Statistics (editor.statistics.Statistics)**

- 记录每个文件在当前 Session 中的编辑时长。
- 由 Workspace 在文件切换与关闭事件中驱动。
- 提供：
 - getEditTime(filePath)：返回毫秒。
 - formatDuration(milliseconds)：转换为“X 秒 / X 分钟 / X 小时 Y 分钟 / X 天 Y 小时”。

- **SpellChecker 系列 (editor.spellcheck.*)**

- SpellChecker 接口：抽象拼写检查服务。
- SpellError：封装单个拼写错误信息。
- LanguageToolSpellChecker：
 - 适配器实现，内部用简单词典模拟 LanguageTool。
 - 将第三方库/API 依赖与上层解耦。

- **Logger / EventPublisher / WorkspaceMemento**

- 继承 Lab1 设计：
 - EventPublisher + EventListener 实现观察者模式。

- `Logger` 订阅命令事件并写入日志文件。
- `WorkspaceMemento` 与 `WorkspaceState` 管理工作区状态的持久化。

2.1.3 模块依赖关系

Main

```

└> CommandParser
└> CommandExecutor
|   └> Workspace
|       └> Editor 接口
|           └> TextEditor
|               └> XmlEditor
|           └> Logger
|           └> WorkspaceMemento
|           └> Statistics
|       └> EventPublisher
|           └> EventListener(Logger)
|   └> SpellChecker (接口)
|       └> LanguageToolSpellChecker (适配器实现)
|

```

TextEditor

```

└> Command (Append/Insert/Delete/ReplaceCommand)

```

XmlEditor

```

└> XmlElement (组合结构)
    └> XML 命令对象 (InsertBefore/AppendChild/EditId/EditText/DeleteElement)

```

Statistics

```

└> 被 Workspace 调用 (文件切换/关闭)

```

SpellChecker

```

└> 被 CommandExecutor 调用 (spell-check 命令)

```

2.2 核心设计

2.2.1 设计模式应用说明

• 命令模式 (Command Pattern)

◦ 应用位置：

- 文本命令： `editor.editor.command.*` (Append/Insert/Delete/Replace)。
- XML 命令： `InsertBeforeCommand` , `AppendChildCommand` , `EditIdCommand` , `EditTextCommand` , `DeleteElementCommand` 。

- 作用：
 - 将编辑操作封装为对象，通过 `Editor.executeCommand` 统一处理。
 - 在 `TextEditor / XmlEditor` 内维护撤销栈和重做栈，支持 `undo/redo`。
 - 易于扩展新命令，符合开闭原则。
- **组合模式 (Composite Pattern)**
 - 应用位置：
 - `XmlElement + XmlEditor`。
 - 作用：
 - 用统一的元素类型表示树中所有节点（叶子和非叶子）。
 - 支持递归遍历、树形打印、递归删除等复杂结构操作。
 - 简化 XML DOM 树的表示和操作。
- **观察者模式 (Observer Pattern)**
 - 应用位置：
 - 事件与日志：`EventPublisher + EventListener + Logger`。
 - 作用：
 - `CommandExecutor` 只负责发布命令事件，不关心日志实现。
 - `Logger` 订阅事件并写入日志文件。
 - 解耦业务逻辑与日志，便于未来扩展其他观察者（统计/审计）。
- **备忘录模式 (Memento Pattern)**
 - 应用位置：
 - 工作区状态：`WorkspaceMemento + WorkspaceState`。
 - 作用：
 - 封装工作区状态的保存/恢复逻辑。
 - 对外只暴露保存/加载接口，不暴露内部数据结构。
- **适配器模式 (Adapter Pattern)**
 - 应用位置：
 - 拼写检查模块：`SpellChecker` 接口 + `LanguageToolSpellChecker` 实现。
 - 作用：
 - 隔离第三方拼写检查库或 HTTP API 细节。
 - 上层只依赖接口，便于替换真实实现或使用 Mock 进行单元测试。

2.2.2 其他设计相关说明

- **多态与类型判断**
 - `Workspace` 抽象为 `Editor`，对外不暴露具体类型。
 - `CommandExecutor` 在需要区分文本/XML 操作时使用 `instanceof` 做安全的类型分派。
- **错误处理与鲁棒性**
 - 所有命令在执行前都会检查参数合法性和上下文（如是否有活动文件）。
 - 针对 XML 的 ID 唯一性、根元素不可删除/改 ID、禁止混合内容等都在相应操作中进行检查并以中文错误提示。
 - 拼写检查和统计模块出错时只打印警告，不阻塞编辑功能。

- **编码与跨平台**
 - 所有文件 IO 统一使用 UTF-8。
 - XML 序列化时添加标准 XML 声明，保证兼容性。

2.3 运行说明

2.3.1 编程语言及版本

- **编程语言**：Java
- **版本**：Java 11
- **构建工具**：Maven (3.6+, 开发环境为 3.9.9)
- **测试框架**：JUnit 5 (5.9.2)

2.3.2 安装依赖的步骤

1. 安装 JDK 11

- 安装 JDK 11，并配置 `JAVA_HOME` 和 `PATH`。
- 终端执行 `java -version` 确认版本。

2. 安装 Maven

- 安装 Maven，配置 `MAVEN_HOME` 和 `PATH`。
- 终端执行 `mvn -v` 确认安装成功。

3. 下载项目依赖并编译

- 在项目根目录执行：

```
mvn compile
```

2.3.3 运行程序的命令

在项目根目录下：

- 使用 Maven 运行：

```
mvn exec:java -Dexec.mainClass=editor.Main
```

`pom.xml` 已配置 `exec-maven-plugin` 的 `mainClass`，也可以简写为：

```
mvn exec:java
```

- 如果已经打包 jar (例如 `mvn package` 后)：

```
java -jar target/text-editor-1.0.0.jar
```

程序启动后进入命令行界面，可使用 Lab1 + Lab2 的全部命令：

- 工作区命令：
 - load <file>
 - save [file|all]
 - init <text|xml> <file> [with-log]
 - close [file]
 - edit <file>
 - editor-list
 - dir-tree [path]
 - exit
- 文本编辑命令（仅限文本编辑器）：
 - append "text"
 - insert line:col "text"
 - delete line:col len
 - replace line:col len "text"
 - show [start:end]
- 日志命令：
 - log-on [file]
 - log-off [file]
 - log-show [file]
- XML 编辑命令（仅限 XML 编辑器）：
 - insert-before <tag> <newId> <targetId> ["text"]
 - append-child <tag> <newId> <parentId> ["text"]
 - edit-id <oldId> <newId>
 - edit-text <elementId> "text"
 - delete-element <elementId>
 - xml-tree [file]
- 拼写检查命令：
 - spell-check [file]

2.3.4 运行测试的命令

在项目根目录执行：

```
mvn test
```

Maven 会自动下载测试依赖并运行所有 JUnit5 测试用例，测试报告输出到 target/surefire-reports/ 目录。

2.4 测试文档

2.4.1 测试用例列表

- **init 与工作区相关命令**

- `init` (Lab1 旧格式):
 - `init test.txt`: 创建文本缓冲区, 标记为已修改。
 - `init test.txt with-log`: 首行写入 `# log`, 自动启用日志。
- `init` (Lab2 新格式):
 - `init text textFile.txt`: 显式指定文本类型, 创建 `TextEditor`。
 - `init xml config.xml / init xml config.xml with-log`: 创建 `XmlEditor`, 根元素为 `<root id="root">`, 带 `with-log` 时根元素包含 `log="true"`。
- `load/save/save all/close/edit/editor-list`:
 - `load`: 将磁盘文件加载到工作区。
 - `save`: 保存当前活动文件, 并清除修改标记。
 - `save all`: 保存所有已修改文件。
 - `edit`: 切换活动文件。
 - `editor-list`: 列出当前打开文件, 显示活动标记、修改标记和会话内编辑时长。

- **文本编辑命令 (Lab1)**

- `append`: 在末尾追加一行文本。
- `insert`: 在指定行列插入文本, 支持换行符插入。
- `delete`: 从指定行列起删除指定长度字符。
- `replace`: 在指定位置删除并插入新文本。
- `show`: 按行号范围显示内容。
- `undo/redo`: 撤销/重做最近的文本编辑操作。

- **日志命令 (Lab1)**

- `log-on / log-off`: 启用 / 关闭指定文件的命令日志。
- `log-show`: 显示当前文件的命令执行日志。

- **XML 编辑命令 (Lab2 新增)**

- `insert-before <tag> <newId> <targetId> ["text"]`: 在同级元素中将新元素插到 `targetId` 元素之前; 测试用例检查新 ID 插入顺序正确, ID 不重复, 根元素前插入时报错。
- `append-child <tag> <newId> <parentId> ["text"]`: 在指定父元素末尾追加子元素; 测试用例检查父元素下多出一个子元素, 父元素有文本时触发“混合内容”错误。
- `edit-id <oldId> <newId>`: 修改元素 ID; 测试用例验证非根元素 ID 成功更新, `idMap` 中不再包含旧 ID, 且尝试修改根元素 ID 时给出错误提示。
- `edit-text <elementId> "text"`: 修改元素文本; 测试用例验证无子元素节点文本更新成功, 有子元素时报“该元素有子元素, 不支持混合内容”。
- `delete-element <elementId>`: 删除指定 ID 元素及其所有子树; 测试用例验证普通元素被完全删除, 尝试删除根元素时报“不能删除根元素”。
- `xml-tree [file]`: 以树形结构打印 XML; 测试用例验证在当前活动 XML 文件上执行命令不会抛异常, 输出包含根节点和子节点层级结构。

- **拼写检查命令 (Lab2 新增)**

- `spell-check [file]` :
 - 文本文件：对全部文本内容进行拼写检查，测试用例包含常见错拼（如 `recieve`），验证输出中有行号、列号和建议拼写。
 - XML 文件：仅检查元素文本内容，测试用例在元素文本中设置错拼（如 `Itallian`），验证输出中包含元素 ID 和建议单词。
 - 异常情况：未打开文件或文件未在工作区时，输出对应错误提示；内部异常时输出警告而不中断程序。

2.4.2 测试执行结果

- 使用 `mvn test` 运行所有自动化测试用例：
 - `CommandParserTest`：覆盖 Lab1 + Lab2 所有命令格式的解析，包括 `init` 新旧两种格式、XML 编辑命令和 `spell-check` 等。
 - `CommandExecutorTest`：覆盖对应命令的执行路径，验证对编辑器内容的实际影响（文本和 XML 两种类型）、统计显示、日志调用和错误处理。
 - 其它测试类（`TextEditorTest`、`TextEditorNewlineTest`、`WorkspaceTest`、`LoggerTest` 等）验证底层编辑器、工作区、日志和事件发布模块的行为。
- 在当前代码版本下，`mvn test` 全部通过，说明：
 - Lab1 的 18 个命令在现有实现下保持功能正常。
 - Lab2 修改的命令（`init`、`editor-list`）和新增的 7 个命令均已在解析与执行两个层面通过测试验证。
 - XML 解析/序列化、ID 管理、撤销/重做、统计模块与拼写检查等新增模块协同工作正常，不影响原有文本编辑功能。